

CHOWLY

A dining platform for The Golden Gate, Lagos

Deborah Akinbola

Repository <https://github.com/DebbyA007/CHOWLY>

Deployed <https://chowly-theta.vercel.app>

This document answers the three deliverables of the CHOWLY build assignment in the order the brief lists them. It is written to be read without any other document open.

Contents

Deliverable 1: The git repository	3
Deliverable 2: The URL of the deployed application	3
Deliverable 3: The document	3
1. How it was built	3
The stack	3
The structure	4
The data model as it was finally implemented	4
How the application was deployed	6
2. How AI was used	7
The tools	7
What it was asked to do	7
What was accepted	7
What was rejected	7
What had to be corrected by hand	8
3. The specific behaviour of the application, from menu to payment	9
Menu browsing	9
Order placement	10
The wait, and the way it is shown	10
Complaint and rating	10
Order assignment	11
Payment	11
Cancelling an order, which is beyond the requirements	12
The two roles	12
Real storage, and what happens on refresh	12
Two more things built beyond the requirements	12
4. How to use it: a walkthrough a stranger can follow	13
A recorded walkthrough	14
Where each requirement is	14

Deliverable 1: The git repository

The codebase is at <https://github.com/DebbyA007/CHOWLY>. It is public, so facilitators need no invitation to read it.

The commit author name reads Deborah Akinbolade throughout the history. That is a typo in the git configuration; the correct name is Deborah Akinbola.

The commit history is the work as it was done, not a tidy retelling of it. Nothing was squashed and no history was rewritten. The repository therefore contains commits that fix things earlier commits broke, and their messages say so. Three worth opening, because they show the method rather than the result:

- "fix: placing an order was returning 400 on production". A commit had added a field the client needed to the request body it posts. The order endpoint validates strictly, so it refused every order with "Unknown field: foodIds". Typecheck, lint, thirty tests and a build were all green, because nothing in any of them compared the body the client sends with the schema the server accepts. The fix separates the two and adds a test that asserts the wire shape both ways.
- "feat: replace the menu with The Lagos Table card, and four model deltas". The menu was replaced late, from ninety two rows in the assignment's own menu document, and four changes to the data model came with it.
- "feat: refuse to write to the production database by mistake". Development and production shared one database, a reseed on a branch degraded the live link, and this is the guard that stops it happening again.

The repository also carries the assignment brief and the engineered model it was built from, at docs/assignment/, so a reader can check the work against the thing it was marked on without leaving the repository.

Deliverable 2: The URL of the deployed application

<https://chowly-theta.vercel.app>

It opens on the front door. There is nothing to install and nothing to sign in to, and both roles are reachable from that first screen. Adding ?table=12 to the URL is what a QR code on a physical table would carry; without it the door asks for the table number.

Deliverable 3: The document

The four required parts follow in the order the brief lists them.

1. How it was built

The stack

Layer	Choice	Why
Framework	Next.js 15, App Router, TypeScript strict	One project serves the pages and the API, so the types the server computes are the types the client renders
Database	PostgreSQL on Neon	Provisioned through the Vercel marketplace, so the deployment and the database are one setup step
ORM	Prisma 6	Migrations are files in the repository, and the generated client makes a schema change a compile error rather than a runtime one
Validation	Zod	Every request body is parsed by a strict schema before anything touches the database
Styling	Tailwind v4	Design tokens live in one stylesheet and nothing else defines a colour
Motion	anime.js v4	Scoped animations that clean themselves up under React strict mode
Data fetching	SWR	Polling with the previous data kept on screen, so nothing blinks while it revalidates
Hosting	Vercel	Deploys from the default branch on push

The brief says the stack is not what is marked, so these were chosen to keep the work honest rather than to be interesting. The one that earned its place repeatedly is Zod: because every route rejects an unknown field, a client bug that would otherwise have corrupted an order arrived instead as a 400 naming the field.

The structure

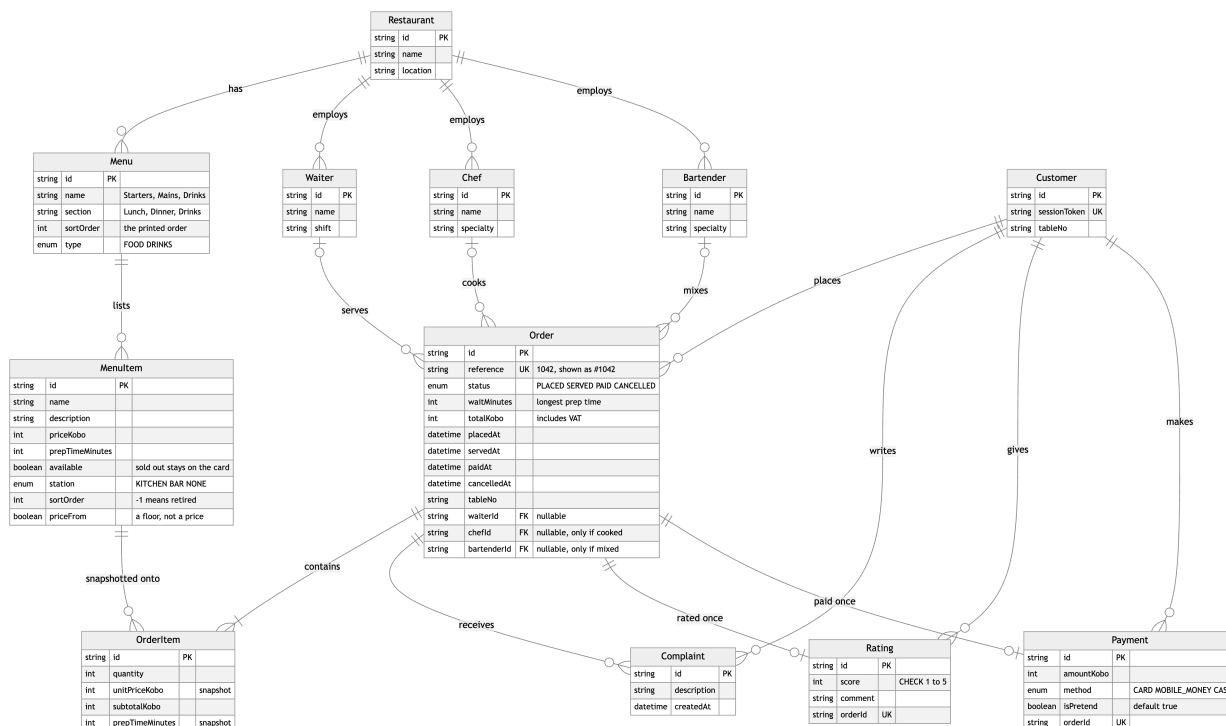
- `app/` holds the routes. Pages under `app/(guest)` and `app/waiter`, and the API under `app/api`. Every route handler is a few lines: parse, authorise, compute, respond.
- `lib/` holds everything that decides something. The wait time, the money, the session token, the cart, the greeting, the cancel window, the staff PIN comparison and the database guard are each one file with unit tests beside it. Nothing in `lib` imports a React component.
- `components/night/` holds the screens. They are named for what they show: menu, order, pay, waiter, receipt, vessel.
- `prisma/` holds the schema, the migrations and the seed.
- `docs/` holds this document, the longer working document it draws on, the AI log, the photography record and the design handoff.

The rule the structure follows is that a number a guest sees is computed in exactly one place on the server. The promised wait is computed in `lib/wait-time.ts` and nowhere else; the money in `lib/money.ts` and nowhere else. The client is allowed to draw a countdown, but not to decide one.

The data model as it was finally implemented

All twelve entities of the engineered model are implemented, and every foreign key it lists is present, including the customer key on Complaint, Rating and Payment. The schema departs from

the model in fifteen deliberate ways. Each is annotated DELTA in prisma/schema.prisma beside the field it changes. The brief asks that a change forced by the build be made and explained, so here they are, grouped by what forced them.



The data model as implemented. Every entity of the engineered model is present; the fields added or changed by the fifteen deltas are visible on Menu, Menuitem and Order.

Correctness of money and time, which the model could not express:

1. Menuitem.prepTimeMinutes exists. The brief requires it and the promise is computed from it.
2. Order.waitMinutes is an integer. The model stored a string like "25 mins", which cannot be compared to a clock.
3. placedAt, servedAt and paidAt are timestamps. The model split date and time across two columns, which cannot be sorted or compared.
4. All money is integer kobo. No floating point value exists anywhere near a total. VAT at 7.5% is added to the stored total and the subtotal is read back from the lines, so no figure is stored twice.
5. OrderItem.unitPriceKobo and prepTimeMinutes are snapshots taken when the order is placed, so editing the menu never rewrites a historical order or the payment taken against it.

Things the model asserted that are not true of a restaurant:

1. Order.waiterId, chefId and bartenderId are nullable. A guest submits with no staff attached and the waiter records them afterwards. NOT NULL would make requirement 3 impossible.
2. Menuitem.station is KITCHEN, BAR or NONE, and the staff an order needs is derived from it. The model gives every order a chef and a bartender. Still and sparkling water are poured, not prepared, so an order of water needs neither, and naming one would put a name on a receipt for work nobody did. Fifty one dishes are kitchen, thirty eight are bar and three are neither.

3. `MenuItem.priceFrom`. Three bottles on the card are listed "from 75,000" and have no fixed price. Storing the floor and charging it would produce a quietly wrong bill, so the row shows the floor, says it is a starting price, and the order endpoint refuses it.
4. `Menu.section` and `Menu.sortOrder`, plus `MenuItem.sortOrder`. A printed card has two levels, a heading such as Lunch and sub-headings such as Starters, and an order that is not alphabetical. One flat name could carry neither. A `sortOrder` of -1 means retired: off the card, still in the database, so an order placed months ago still reads back in full.

States the model had no way to record:

1. `OrderStatus` is PLACED, SERVED, PAID and CANCELLED. Late is not a status. It is derived at read time from `placedAt` and `waitMinutes` and never stored, so there is no row to hand-set to make the complaint flow work.
2. CANCELLED and `Order.cancelledAt`. The model's statuses only ran forward, so a guest who ordered by mistake had nothing to do but wait. A guest may now withdraw their own order inside the first quarter of the promised wait.

Integrity the model left to the application:

1. `Payment.orderId` is unique and the insert and the status change run in one transaction, so a double-tapped button records one payment.
2. `Payment.isPretend` defaults to true and is shown on the button and on the receipt.
3. `Rating.orderId` is unique, and CHECK (score BETWEEN 1 AND 5) is added by a hand-written migration, because Prisma cannot express it. Zod validates first; the database is the last line.
4. `Customer.sessionToken` is unique, since there are no logins and the token is the identity.

How the application was deployed

Vercel imports the repository and deploys the default branch on every push. Three settings matter.

- The install command is `npm ci --ignore-scripts`, so no package's install script runs on the build machine.
- Because that also blocks Prisma's own code generation, the build command is `prisma generate && next build`. It does not run migrations: a deploy that silently changes the database schema is a worse problem than the one it solves.
- `DATABASE_URL` is Neon's pooled connection string and `DIRECT_URL` is the direct one, which migrations need. No secret is prefixed `NEXT_PUBLIC_`, so nothing from the environment reaches the browser bundle.

Migrations are applied deliberately, with `prisma migrate deploy` pointed at production, as a release step rather than a side effect of a deploy.

One deployment incident is worth recording because it changed how the project is set up. Development and production shared one Neon database. Reseeding it from a feature branch replaced the menu everywhere at once, and the deployed app, still running the old code, dropped to six rows with four of them marked sold out. Nothing was corrupt and no order lost its lines, but the live link was degraded until the branch merged. The lesson is not to be careful with the seed: with one database, a data change on a branch is a production change. Development now points at its own Neon branch, and anything destructive compares the host it is aimed at against

PRODUCTION_DB_HOST and refuses to touch it. The steps and the guard are in docs/DATABASE.md.

2. How AI was used

The tools

Claude, used as a pair programmer inside Claude Code, the terminal client that can read and write files in the repository and run commands. It was used for the whole build, not for a part of it: schema, routes, screens, animations, tests, seeding, photograph sourcing, and this document. A running record of what was asked, accepted, rejected and corrected was kept in docs/AI-LOG.md as the work happened, not reconstructed afterwards. It is roughly two and a half thousand lines and is the honest version of this section.

What it was asked to do

The instructions it worked under are committed at CLAUDE.md, so what it was told is auditable rather than described. They are specific and they are constraints rather than requests: money is integer kobo, every route validates with a strict Zod schema, the customer identity is a signed httpOnly cookie and never localStorage, ownership is checked inside the query on every route that reads or changes an order, no em dashes anywhere, no screen shake or flashing in any animation.

The work was given in whole features rather than snippets: "build the ticket rail", "replace the menu with this document's ninety two dishes and tell me what that forces in the model", "the guest may cancel inside a window, computed on the server". Each came back as code plus a set of claims, and the claims were then checked.

What was accepted

- The shape of the codebase. One computation per file in lib, thin route handlers, screens named for what they show. It held for the whole build.
- The derived-delay idea. Late is computed from placedAt and waitMinutes rather than stored. It removes a column that could disagree with the clock, and it is why the complaint can be gated honestly.
- The station field and the staff it implies, which is delta 12. It came from asking why a bottle of water needs a bartender.
- The security posture. Prices never trusted from the client, payment idempotent through a unique constraint and a transaction, rate limits counted in the database, ownership inside the query.
- Almost all of the copy, after correction. The rule that an error says what went wrong and how to fix it, and never apologises, produced better sentences than the first drafts of them.

What was rejected

- Storing a delayed flag on the order. Two sources of truth for one fact, and the one in the database would be the stale one.
- Storing a total sent by the client, and later a subtotal alongside the total. Both were rejected for the same reason: a figure stored twice is a figure that can disagree with itself.

- A waiter cancel button. It is the obvious companion to the guest's cancel and a real restaurant would have one, but on a surface where the waiter side opens with one tap it is authorised by nothing. It would be the only destructive action a stranger with the link could take against someone else's table, and unlike marking an order served it could not be undone by the table.
- Thirteen category chips in one horizontal scroller for the new menu. That is a list of tabs, not a menu. The card has seven printed headings and the sub-headings sit under them.
- An "In the kitchen" step in the order tracker. Invented from nothing the database records. It was removed.
- Three earlier art directions, built as working prototypes and then discarded on their merits. They are still in the repository at /directions, because a design decision with the rejected options attached is worth more than one asserted.
- Roughly two thirds of the photographs sourced for the menu. The searches returned an engraving of a lobster for lobster pepper soup, a portrait for the tasting menu, a nude torso for a mango spritz and six photographs of a concert for suya, because a band shares the name. Every one of those was thrown out by looking at it.

What had to be corrected by hand

This is the part that is worth reading, and the log records more of it.

- The production outage described under deliverable 1. A field the client needed was added to the request body it posts, and the strict schema refused every order. Four green checks did not see it, because none of them compared the client's body to the server's schema. Corrected by separating the two types and adding a test that asserts the wire shape in both directions.
- The entity relationship diagram in the README, which went three schema changes out of date without a single check going red. It still showed a status of PLACED, SERVED and PAID with no CANCELLED, and had no station, no priceFrom, no section and no sortOrder. Four migrations shipped over a picture that no longer showed the model, and it was only noticed while rendering the diagram for this document. It is the same failure as the outage above on a different surface: the checks measured everything except the thing that was wrong. A diagram is documentation that no test reads.
- The photograph search, where the obvious query was the wrong one. Two hundred and eight candidates were collected for the fifty dishes that still lacked a photograph, using the menu's English names, and five of those dishes came back with nothing usable at all. One further pass on the Nigerian names alone finished the card in a single run: moi moi steamed in its leaves, egusi beside a ball of swallow, akara straight out of the pan, ofada rice served on leaves. "Bean pudding" and "melon seed soup" never returned anything, and "moi moi" and "egusi" returned the dish immediately. The correction was to the instruction rather than to the code: search in the language the food is named in.
- A review method error. A panel of checks was run over the codebase and repairs were started while the verifiers were still reading the same working tree, so several findings were dismissed as "already correct" when they had in fact just been fixed. The numbers from that pass are not evidence and the log says so. What the fixes rest on is before-and-after measurement done afterwards, one at a time.

- An animation library used from memory of its previous major version. anime.js v4 renamed easing to ease, changed direction: 'alternate' to alternate: true, and dropped the default export. Code written from v3 habits runs and silently does nothing. Corrected against the installed version's own types.
- A security flag that failed the wrong way. The staff PIN check was written as on unless the flag is exactly false, which locked the waiter side of the production deployment where the variable had never been set. It is now on only when the flag is exactly true, and its tests cover every shape of the value.
- Copy that assumed a kitchen. Sentences on the order screen said "it is with the chef now" for an order of still water. Three of them now name the kitchen, the bar or the waiter, from what is actually on the order.
- "Promised in 1 minutes". Every dish on the earlier card took longer than a minute, so a hand-written plural had never been wrong. Corrected to use the helper that has always known the singular.
- A three pixel layout shift. When the cancel window closed, the block holding the button was two pixels shorter without it, so the content below moved. Found by measuring the position of an element every two seconds across the close, in both browsers, rather than by looking.

The pattern across all of them is that the AI was reliable at structure and at following a written constraint, and unreliable at anything requiring the world outside the code: what a browser does at a real size, what a photograph is of, what a plural should be when the data changed under it. Everything in this project that was checked by running it and looking at it held. Everything that was checked by reading the code did not.

Two of those were caught by a check that did not exist, so the check now does. lib/schema-diagram.test.mts reads prisma/schema.prisma and the diagram in the README and fails if an entity is missing from the picture, if an enum value in the schema is absent from it, or if a field introduced by one of the fifteen deltas does not appear. Run against the README as it actually stood before this was noticed, three of its four assertions fail. It deliberately does not require every field to be drawn, because a diagram is allowed to leave out a restaurant's phone number; it requires the parts of the model this document is about. The photograph search has no equivalent and stays a manual check: no test can tell whether a picture is of the dish, and looking at every one of the four hundred and thirty eight candidates at the size it would be shown is what did it.

3. The specific behaviour of the application, from menu to payment

The five features the brief names are covered first, each under its own heading, then the ones built beyond them.

Menu browsing

A guest opens the link at a table. The front door shows the room, the restaurant's name and address, "I'm a guest" and "I'm a waiter", and the table number the link carried or a field asking for it.

Tapping "I'm a guest" opens the card. Under the restaurant's name is a greeting set by the hour on the guest's own device: "Good morning. What would you like this morning?", changing at five, at

noon and at five again. It uses no honorific, because with no login the app knows nothing about who is holding the phone.

The card is the restaurant's printed menu: ninety two dishes across seven headings, with a lighter second row of sub-headings under any heading that has them, so Lunch offers Starters, Mains and Drinks. Tapping either filters in place. Each row is a 76 pixel round photograph, the dish name, its description, its price in naira and the kitchen's preparation time in minutes. A description longer than three lines is clipped with "Show all of it" under it, because the tasting menu lists eight courses.

Two states a guest meets on the card. A dish the kitchen has run out of stays on the card, greyed, with "Sold out" where the add control was, so a guest can see the restaurant has it. Three bottles are listed from a floor price: they read "from N75,000" and carry "Ask your waiter" in the same place, because there is no price yet to put on a bill. The menu refreshes every thirty seconds and when the tab regains focus, so a dish the kitchen takes off greys without a reload.

Order placement

The ochre circle on a row adds the dish and morphs into a stepper with minus, the count and plus. The cart bar at the bottom never disappears: empty it says "Your order is empty", and with items it shows the count and the running total, which tick rather than jump.

"View order" opens the order sheet: the lines with their steppers, the subtotal, VAT at 7.5%, the total, and the table number. "Place order" sends the menu item ids and quantities and the table number, and nothing else.

What the server does with that is the important part. It reads the price and the preparation time of every requested dish from the database, snapshots them onto the order lines, computes the subtotal from the lines, adds VAT, and computes the promised wait as the longest preparation time in the order. A request carrying a price is rejected with a message naming the field. A request carrying a dish that sold out while it sat on the screen is refused by name, and the client takes it off the order and says what changed.

The Order tab opens immediately with a provisional order drawn from the same formula the server uses, so the guest is not looking at a spinner, and the kitchen's real order replaces it when it lands. The order screen then shows the order number, the table, the items, and the wait.

The wait, and the way it is shown

The wait is the spine of the story the brief describes, so it is the centre of the screen rather than a line of text. A 184 pixel ring empties as the promised minutes are used, recomputed every second from `placedAt` and `waitMinutes`, so a refresh lands exactly where the clock is and no client-side counter can drift. The numerals inside count down in tabular figures. Above the ring is a vessel that matches the order: a pot simmering for anything cooked, a glass being poured for a drinks-only order.

When the promise is spent, the screen crosses from ochre to a late red over about two minutes, slowly, never a snap and never a flash. The ring closes and the numerals count up. Late is derived, not stored: it is now $> \text{placedAt} + \text{waitMinutes}$ while the order is still placed.

Complaint and rating

The complaint is earned rather than always present. "Report a problem" appears only once the order is actually late, which is what the brief describes, and the server enforces the same rule: a complaint on an order that is not late is refused, so a hand-made request cannot bypass the screen. The guest writes a line and sends it; it is stored against the order and against the customer, and the waiter's list shows the count against that table within three seconds.

"Rate your order" takes one to five and an optional comment. One rating per order is enforced by a unique constraint, and rating again changes the existing one rather than adding a second. The score range is checked by Zod and again by a database CHECK constraint. Both the complaint and the rating come back on the waiter's copy of the order.

Order assignment

The waiter side opens with one tap from the front door and no prompt. The rail polls every three seconds and keeps the previous list on screen while it revalidates, so nothing blinks. Each order is a card with its table and number, its items and total, and a status with its own dot and time: Just placed, In the kitchen with the time left, Ready in the last minute, Late with the minutes over in red, or Served. All, Cooking, Late and Served filter the list.

Opening an order shows when it was placed, its clock, its lines with their minutes, and any complaint or rating from the table in full. Then the staff, and which pickers appear is derived from the order rather than assumed. Chef appears only if something on the order is cooked, and Bartender only if something is mixed. A dish and a cocktail show all three; a round of cocktails shows two; an order of still water shows the waiter alone, under a line that says nothing on it is cooked or mixed.

"Mark as served" records the staff and sets the served timestamp. The screen shows it immediately and puts the order back the way it was if the server refuses. The endpoint derives the same answer from the order's own lines, so it refuses a missing chef for a cooked order and equally refuses a chef sent for an order with nothing from the kitchen. Only a placed order can be served; serving twice is refused rather than silently rewritten.

The guest's screen picks the change up on its next poll, within three seconds and with no reload: the ring reads Served with the time, the vessel becomes the plated dish or the full glass, and the screen leads with the way to settle.

Payment

The Pay tab shows the bill: the lines, the subtotal, VAT and the total, and three ways to pay, which are card, bank transfer and cash at the till. The button reads "Pay N4,838 (pretend)" and the word pretend is on the button and on the receipt, because the brief requires the payment to be recorded and clearly labelled as pretend.

The payment is written inside one database transaction that inserts the payment row and moves the order to PAID together, and Payment.orderId is unique. A double-tapped button therefore records one payment, not two. Payment.isPretend defaults to true in the schema, so a pretend payment is pretend in the data and not only in the wording.

The receipt then prints: a perforation under the header, the restaurant and the date, the order number, the table and a receipt number, the lines, the subtotal, VAT, the total struck into the surface, a PAID stamp that lands once, a torn foot, and one credit line naming the staff who

actually prepared it. For an order of still water that line reads "Served by Ada O." and names nobody else.

"Save the receipt" draws the receipt again on a canvas at the phone's own pixel ratio and hands it to the phone. On a phone with a share sheet that is how a picture reaches the photo library, so the line under the button says to tap Save Image; elsewhere it downloads as a PNG, and there is a quiet "Download the file instead" for anyone who wants the file rather than a photo.

Cancelling an order, which is beyond the requirements

A guest may withdraw their own order while two things hold: it is still placed, and no more than a quarter of the promised wait has passed. After that the kitchen has started it and cancelling would throw away food. The window is a fraction of the promise rather than a fixed number of minutes, so it stays proportionate: a glass of water gives fifteen seconds, the chef's tasting menu gives twenty two and a half minutes.

The window is computed on the server from `placedAt` and `waitMinutes` on every request. The button and the countdown beside it are presentation; a tap that leaves the screen inside the window and lands outside it is refused with a sentence saying so. Ownership is part of the query: the order id comes from the path and the customer from the signed session cookie, and neither is ever read from the request body. The endpoint reads no body at all.

When the window closes the button goes away where it stands. The block holding it is a fixed height, so nothing below it moves, and a sentence takes the space. A cancelled order leaves the waiter's live list and the amount owing, and stays reachable for twelve hours, because a waiter may have it open when the table cancels it.

The two roles

The role switch is the front door: two buttons, one tap each, and a tab bar on each side. Tapping the CHOWLY lockup at the top of any screen returns to that door. There are no logins anywhere, which is what the brief allows, and this document says so rather than presenting a PIN prompt as security.

What a guest can do is still bounded. Their identity is an opaque token in a signed, `httpOnly`, `sameSite` cookie, never in `localStorage` where injected script could read it. Every read, complaint, rating, cancellation and payment checks ownership inside the database query, so one table cannot read or pay another's order. Order creation, complaints and ratings are rate limited per session, counted in the database. The server-side authorisation seam for the waiter routes exists and is tested, behind a flag that is off by default, because a deployment that never set the variable must not lock its own waiter side.

Real storage, and what happens on refresh

Everything is in PostgreSQL: the restaurant, the menus, the dishes, the staff, the customers, the orders and their lines, the complaints, the ratings and the payments. Refreshing any screen changes nothing a guest can see. The countdown in particular is computed from the stored `placedAt` rather than from when the page loaded, so a refresh puts the ring exactly where it was.

Two more things built beyond the requirements

- The 86 board. The waiter's Menu tab lists every dish with a switch. Taking one off removes it from the guests' card within the minute and refuses by name any order still carrying it.
- The table board. The waiter's Tables tab is the floor by table, with every order of the last twelve hours and what each table still has to pay, and the night's outstanding total.

4. How to use it: a walkthrough a stranger can follow

You need a phone or a browser and the link. Nothing to install and nothing to sign in to. The whole pass takes about five minutes.

1. Open <https://chowly-theta.vercel.app/?table=12>. The dining room, the name, and "You're at table 12". Opening it without ?table=12 makes the door ask for the number instead.
2. Tap "I'm a guest". The card opens on Breakfast, with the greeting for whatever hour it is where you are. The chips along the top are the seven printed headings; under a heading with sub-headings of its own, a lighter second row carries them.
3. Tap Drinks, then Wines. The three bottles priced "from N75,000" have no add control and say to ask your waiter, because there is no price yet to put on a bill.
4. Tap Drinks, then Soft, and add one Premium Still Water with the ochre circle, and nothing else. This is the one thing to get right if you are short of time. The promise is the longest preparation time in the order, water takes one minute, so the order runs late in one minute. That is the only way to see the complaint, which the app offers once an order is late and not before. A jollof rice would keep you waiting twenty two minutes for the same thing.
5. Tap "View order", check the table number, and tap "Place order". The Order tab opens at once and the kitchen's order number lands on it a moment later.
6. Watch the glass and the ring. Under them is "Cancel this order" with the time you have left counting down beside it: fifteen seconds for water. Watch it reach zero. The button goes away where it stands, nothing below it moves, and a sentence takes its place. If you would rather use it than watch it expire, place a second order and tap it inside the window; it asks once, then the screen becomes the cancelled order.
7. Wait for the promise to run out. The ring closes, everything ochre crosses slowly to red, and "Report a problem" and "Rate your order" appear. Tap Report, write a line, send it. Then tap Rate, pick a low number, and send that too.
8. Switch to the waiter. Tap the CHOWLY lockup at the top to go back to the door, then "I'm a waiter", or open <https://chowly-theta.vercel.app/waiter> directly. There is no PIN and no prompt. Your table is in the list, marked Late, with the count of your report.
9. Tap the pill, "Who's serving?", and pick a name; it is kept for the session. Tap the card. Your report is there under "From the table". Notice what the screen asks for: an order of water asks for a waiter and nobody else, and says why. Tap "Mark as served".
10. Go back to the table: the lockup, then "I'm a guest", then the Order tab. Within three seconds and with no reload it reads Served with the time. Tap Pay.
11. Pick a method and tap "Pay ... (pretend)". The receipt prints, with the stamp, the torn foot, and a credit line naming only the waiter, because nothing on that order was cooked or mixed. Tap it twice if you like: the record is one payment. "Save the receipt" hands the picture to your phone.

12. Two more things worth a minute. On the waiter side, the Menu tab is the 86 board: switch a dish to "Sold out", then look at the guests' card and find it greyed with a tag. And turn on Reduce Motion in your system settings and reload: every entrance becomes a fade, and the late colour change still happens, slower, because it is a colour and not a movement.

A recorded walkthrough

The same pass is recorded end to end at docs/media/walkthrough.mp4 in the repository: three minutes forty five, one continuous take on the deployed app, no cuts and no second tab.

Order #1009 in the live data is the order that recording pays for, kept deliberately so the walkthrough can be checked against it; only its receipt number differs, reading 0006 now rather than the 0007 in the video, because that number is the payment's place in the sequence of payments and the test payments recorded around it have since been removed.

Where each requirement is

The brief asks for	Where it is
1. The menu, with a name, a price and a preparation time on every item	Ninety two dishes seeded into PostgreSQL, all three fields on every one. Menu browsing, above
2. Placing an order, with the order details and the waiting time shown	Order placement, above. The wait is computed on the server and shown as the ring
3. Assigning the order: a waiter records the chef and the bartender and marks it served	Order assignment, above. The staff list is seeded; the pickers shown are derived from the order
4. Complaint and rating, both stored against the order	Complaint and rating, above. Both are rows against the order and the customer
5. Payment, recorded, marking the order paid, clearly labelled pretend	Payment, above. One transaction, a unique constraint, isPretend true in the data
6. The two roles, with a simple switch and no logins	The two roles, above. The front door, two buttons
7. Real storage that survives a refresh	Real storage, above. PostgreSQL on Neon; the clock is computed from the stored timestamp
8. A live link anybody can use	https://chowly-theta.vercel.app

Built beyond them: the cancel window, the 86 board, the table board, the receipt as a saveable picture, the derived staff, and the design, which is a single art direction chosen over three prototypes that are still in the repository.